

PR Audit Report

Repository	dipak2-8/demo-git-test-2
PR Number	#full_scan
Risk Level	HIGH

Repository Audit Report — dipak2-8/demo-git-test-2

Repository Overview

- Total security issues found: 4
- Total quality issues found: 0
- GitHub Actions vulnerabilities: 0
- Open PRs analyzed: 0

Repository Issues

Security Issues

- File + Function name + Line number: app/services/auth_service.py — authenticate_user — line 15

```
def authenticate_user(username: str, password: str):
    # simulate DB check
    if not username or not password:
        raise Exception("Invalid input")

    hashed = hashlib.md5(password.encode()).hexdigest() # <- ISSUE HERE
```

```

if hashed == "d41d8cd98f00b204e9800998ecf8427e":
    role = "admin"
else:
    role = "user"

payload = {
    "sub": username,
    "role": role,
    "exp": datetime.utcnow() + timedelta(hours=24)
}

token = jwt.encode(payload, SECRET_KEY, algorithm="HS256")

return token

```

- Why it is dangerous: MD5 is a weak hash and is not suitable for password hashing. It can be cracked quickly, making authentication logic unsafe.
- Fixed version of the full function:

```

def authenticate_user(username: str, password: str):
    # simulate DB check
    if not username or not password:
        raise Exception("Invalid input")

    hashed = hashlib.scrypt(password.encode(), salt=b"static_salt", n=16384, r=8,
                             p=1).hex()

    if hashed == "d41d8cd98f00b204e9800998ecf8427e":
        role = "admin"
    else:
        role = "user"

    payload = {
        "sub": username,
        "role": role,
        "exp": datetime.utcnow() + timedelta(hours=24)
    }

```

```
token = jwt.encode(payload, SECRET_KEY, algorithm="HS256")

return token
```

- Documentation link: https://bandit.readthedocs.io/en/1.9.4/plugins/b324_hashlib.html

-
- File + Function name + Line number: app/services/report_service.py — fetch_external_data — line 9

```
def fetch_external_data(url: str):
    # risky external call
    response = requests.get(url) # <- ISSUE HERE
    return response.text
```

- Why it is dangerous: Requests without a timeout can hang indefinitely, causing resource exhaustion or stalled application behavior if the remote service is slow or unresponsive.
- Fixed version of the full function:

```
def fetch_external_data(url: str):
    # risky external call
    response = requests.get(url, timeout=10)
    return response.text
```

- Documentation link: https://bandit.readthedocs.io/en/1.9.4/plugins/b113_request_without_timeout.html

-
- File + Function name + Line number: app/services/user_service.py — get_user_by_id — line 10

```
def get_user_by_id(user_id: str):
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()

    query = f"SELECT * FROM users WHERE id = '{user_id}'" # <- ISSUE HERE
    result = cursor.execute(query).fetchall()

    conn.close()
    return result
```

- Why it is dangerous: Building SQL with string interpolation allows attacker-controlled input to alter the query, creating a SQL injection risk.
- Fixed version of the full function:

```
def get_user_by_id(user_id: str):
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()

    query = "SELECT * FROM users WHERE id = ?"
    result = cursor.execute(query, (user_id,)).fetchall()

    conn.close()
    return result
```

- Documentation link:

https://bandit.readthedocs.io/en/1.9.4/plugins/b608_hardcoded_sql_expressions.html

-
- File + Function name + Line number: app/services/user_service.py — list_users — line 24

```
def list_users(role: str):
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()

    if role == "admin":
        query = "SELECT * FROM users"
    else:
        query = f"SELECT * FROM users WHERE role = '{role}'" # <- ISSUE HERE

    result = cursor.execute(query).fetchall()

    conn.close()
    return result
```

- Why it is dangerous: Building SQL with string interpolation allows attacker-controlled input to alter the query, creating a SQL injection risk.
- Fixed version of the full function:

```
def list_users(role: str):
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()

    if role == "admin":
        query = "SELECT * FROM users"
    result = cursor.execute(query).fetchall()
```

```
else:
    query = "SELECT * FROM users WHERE role = ?"
    result = cursor.execute(query, (role,)).fetchall()

conn.close()
return result
```

- Documentation link:

https://bandit.readthedocs.io/en/1.9.4/plugins/b608_hardcoded_sql_expressions.html

Code Quality Issues

No code quality issues were reported by the provided findings.

GitHub Actions Vulnerabilities

No GitHub Actions vulnerabilities were reported by the provided findings.

Pull Request Analysis

No open pull requests were analyzed.

Overall Risk Assessment

- Risk Level: HIGH
- Reason: The repository contains multiple security issues, including weak password hashing, SQL injection risks, and an external request without a timeout.